Compression Trade-offs for Offline Skin-Lesion Classification on Edge Hardware

A measured, per-class study of quantization, pruning and quantization-aware training on MobileNetV2 / HAM10000

Vijayabalaguroo S · Independent research · October 2026

Summary

What was done. A MobileNetV2 classifier was trained on HAM10000 (10,015 dermoscopy images, 7 skin-lesion classes) with a lesion-level train/validation/test split. The trained model was then compressed with post-training quantization (PTQ, two kinds), unstructured magnitude pruning at 30/50/70% (each also combined with INT8), and quantization-aware training (QAT). Every variant was measured for file size, single-thread CPU latency, accuracy, and per-class F1 on the same 1,527-image test set, with extra attention to melanoma (mel) and actinic keratosis (akiec), the classes where a miss costs the most.

Main results

- **Baseline:** accuracy 0.685, macro-F1 0.515, mel F1 0.446, akiec F1 0.389 (float32, 8.5 MB). Below the 83% reported by prior MobileNet work, but that comparison is not like-for-like (Section 5).
- **INT8 PTQ cost nothing measurable:** 3.3× smaller (2.6 MB), accuracy 0.713, macro-F1 0.517. Dynamic-range PTQ: 3.5× smaller, macro-F1 0.525. These differences from the baseline are inside sampling noise (about ±2.3 accuracy points).
- **Pruning hurt, and aggregate accuracy hid how much.** At 50% sparsity accuracy was 0.62–0.63 (−5 to −6 points) while akiec F1 fell from 0.389 to 0.036. At 70% the model collapsed (accuracy 0.12). My recovery recipe is the likely cause (Section 4.3), so this says more about the recipe than about pruning.
- **Smaller was not faster on this CPU.** Float32 baseline 6.9 ms; full-INT8 10.5–11.5 ms; dynamic-range INT8 17.7–22.2 ms. Pruning changed neither file size nor latency.
- **QAT (5 epochs) underperformed** (macro-F1 0.359), most likely because it was under-trained.
- **Pareto view:** only the baseline, PTQ INT8 and PTQ dynamic-range are non-dominated on latency vs macro-F1; on latency vs mel F1 only the baseline is. The practical case for PTQ here is size, not speed.

What this report does not show: that quantization is slow in general (one machine, no profiling), that pruning is harmful in itself (the recipe is confounded), or anything about clinical use. It is a measurement study on one dataset, one seed and one split.

1. Motivation and question

Rural clinics and disaster zones are the places where image-based AI would help most and where a reliable connection to a cloud model is least likely. Running a compressed model on the device removes the dependency. Compression is not free, though: it can cost accuracy unevenly across classes, and a smaller model is not automatically a faster one on a given chip.

This project asks, for one concrete task, **what each compression technique costs in accuracy, class by class, and in real milliseconds on a CPU**, instead of relying on parameter counts or FLOPs. Skin-lesion classification on HAM10000 is the test case because it is real clinical data, strongly imbalanced (about two thirds of images are benign nevi, nv), and already has published compression work to compare against.

2. Related work

A literature scan (September 2026) found six directly relevant papers on lightweight models or compression for HAM10000 and closely related skin-lesion data.

#	Paper	What it does	Reported
1	Chaturvedi et al. 2019, arXiv:1907.03220	MobileNet (v1) transfer learning, HAM10000	83.15% accuracy
2	Baig et al. 2023, Diagnostics 13(3):385	ShuffleNet + SE blocks; HAM10000 + ISIC 2019/2020	99.14% accuracy
3	Al Mamun et al. 2025, arXiv:2505.21597	Small custom CNN vs ResNet50, HAM10000	87.05% acc.; 96.7% fewer params; 99.2% fewer FLOPs
4	Wang et al. 2025, arXiv:2507.15958	Ghost + attention net, QAT, spiking version on BrainChip Akida	91.6% top-1; 82.4% macro-F1; 1.5 ms/image on Akida
5	Winata et al. 2026, Springer SCI vol. 1227	Compares compression strategies for edge teledermatology	Distillation best (+2.65%); 109 ms fastest mobile inference
6	Paxton et al. 2025, arXiv:2509.00745	Pruning for fairness, VGG11 / ViT-B16, ISIC 2019	VGG11 79% acc. / 65% F1 after pruning

Table 1. Prior work identified in the scan. Based on abstracts and summaries; two papers (the 2026 Springer chapter and the 2025 ShuffleNet paper) were not read in full.

Compared with these, this report aims to add three things: (i) measured latency on a reproducible single-thread CPU, not FLOPs or one-off hardware, (ii) per-class F1, with mel and akiec tracked explicitly, across every compression level, and (iii) one backbone, one split and one benchmark harness for all techniques. There are two caveats. Paper 5 already compares compression strategies on HAM10000 and reports a measured mobile latency, and I have only seen its abstract; I need to read the full text to see whether it also tracks per-class degradation before treating gaps (ii) and (iii) as settled. Paper 4 reports measured latency too, but on specialised neuromorphic hardware.

3. Method

3.1 Data and split

HAM10000 has several images of the same lesion. Splitting by image would put near-duplicates in both train and test and inflate accuracy, so the split is by lesion ID (GroupShuffleSplit, seed 42, 70/15/15) with an assertion that no lesion appears in two splits. The test set has 1,527 images: akiec 48, bcc 66, bkl 172, df 10, mel 186, nv 1,016, vasc 29. Training has about 7,000 images. A model that always predicted nv would score 66.5% accuracy, so every accuracy figure in this report should be read against that.

3.2 Baseline model

MobileNetV2 (ImageNet weights, 224×224 input) with global average pooling, dropout 0.3 and a 7-way softmax. Augmentation: flips, brightness ±0.1, contrast 0.9–1.1. Phase 1 trains only the new head for 10 epochs at lr 1e-3. Phase 2 unfreezes the backbone from layer 60 at lr 1e-5, for up to 25 epochs with early stopping and best-weights restore. The model is built directly on the backbone's input and output tensors, because nesting the backbone as a layer breaks tfmot's quantize_model later.

Class imbalance is handled with inverse-frequency class weights, plus an extra boost of ×2.0 for mel and ×1.5 for akiec in the final baseline. Three baseline versions were trained:

Version	What changed	Accuracy	Macro-F1	mel F1	akiec F1
v1	Repo script as written: unfreeze from layer 100, 15 fine-tune epochs, inverse-frequency class weights	0.677	n/r	n/r	n/r
v2	v1 + extra class-weight boost (mel ×2.0, akiec ×1.5)	n/r	n/r	n/r	n/r

Version	What changed	Accuracy	Macro-F1	mel F1	akiec F1
v3 (used)	v2 + unfreeze from layer 60, up to 25 fine-tune epochs, early stopping	0.685	0.515	0.446	0.389

Table 2. Baseline development. n/r = not recorded in this report (only the v1 accuracy and the final v3 numbers were kept). v3 was chosen after looking at test-set numbers rather than validation only, so there is a small optimistic bias in the baseline.

3.3 Compression variants

Variant	How it was made	Files
PTQ dynamic-range	TFLite converter, default optimization, no calibration data. Weights stored as INT8; activations stay float.	ptq_dynamic
PTQ full-INT8	Same, plus 100 random training images for calibration; INT8 operators and INT8 input/output.	ptq_int8
Pruning 30 / 50 / 70%	tfmot prune_low_magnitude (element-wise, unstructured), sparsity ramped from 0 to the target over 8 epochs at lr 1e-5, then stripped. Converted exactly like PTQ dynamic-range, so these files hold INT8 weights, not float32.	pruned_30, _50, _70
Pruning + INT8	The stripped pruned model converted with the full-INT8 recipe.	pruned_30_int8, _50_int8, _70_int8
QAT	tfmot quantize_model on the baseline, 5 epochs at lr 1e-5, then converted with default optimization.	qat_int8
Baseline float32	The v3 Keras model converted to TFLite with no optimization; used as the uncompressed reference.	baseline_float32

Table 3. The ten benchmarked variants. Note the pruning and QAT stages used inverse-frequency class weights without the mel/akiec boost, so their recovery training differed from the baseline's.

3.4 Benchmark

Each TFLite file ran through tf.lite.Interpreter with one thread. Latency is the mean of 100 single-image invocations after 5 warm-up calls, on one fixed test image. The whole benchmark was run twice back-to-back in the same Colab session; both runs are reported (Appendix A). Accuracy and F1 come from a full pass over the 1,527 test images. The float32 TFLite baseline reproduced the Keras evaluation exactly (accuracy 0.6850, macro-F1 0.5145), so conversion itself did not change the model. Hardware: the CPU of a Google Colab VM; the exact model and architecture were not recorded.

4. Results

4.1 Baseline and the class-imbalance problem

The final baseline reaches accuracy 0.685 and macro-F1 0.515 (Table 2). The clearest picture of its weakness is the confusion matrix, which I only have for the first (v1) run, so treat it as representative rather than exact: of 186 melanomas, 50 (26.9%) were called nv, and only 85 (45.7%) were recognised.

true \ predicted	akiec	bcc	bkl	df	mel	nv	vasc	n
akiec	28	8	2	0	5	4	1	48
bcc	7	51	1	4	0	2	1	66
bkl	23	24	63	10	34	18	0	172
df	2	0	1	7	0	0	0	10
mel	11	10	11	11	85	50	8	186
nv	7	48	28	38	104	773	18	1016
vasc	0	1	0	0	1	0	27	29

Table 4. Confusion matrix of the v1 baseline (accuracy 0.677), 1,527 test images. The v3 matrix (baseline_v3_eval.json) is not reproduced here.

4.2 All variants

Model	Size (KB)	Smaller	Latency (ms)	Accuracy	Macro-F1	mel F1	akiec F1
Baseline (float32)	8682.7	1.0×	6.9	0.685	0.514	0.446	0.389
PTQ dynamic-range	2447.5	3.5×	17.7	0.687	0.525	0.421	0.421
PTQ full-INT8	2645.6	3.3×	10.7	0.713	0.517	0.431	0.408
QAT (5 ep) -> INT8	2653.0	3.3×	10.5	0.567	0.359	0.239	0.236
Pruned 30% (dyn-range)	2447.5	3.5×	22.2	0.560	0.404	0.334	0.276
Pruned 30% + INT8	2645.6	3.3×	10.9	0.597	0.419	0.333	0.263
Pruned 50% (dyn-range)	2447.5	3.5×	21.9	0.623	0.377	0.229	0.036
Pruned 50% + INT8	2645.6	3.3×	11.5	0.635	0.404	0.322	0.036
Pruned 70% (dyn-range)	2447.5	3.5×	19.6	0.126	0.037	0.000	0.000
Pruned 70% + INT8	2645.6	3.3×	10.5	0.121	0.034	0.000	0.000

Table 5. Size, single-thread CPU latency (mean of two runs), and quality for every variant. "Smaller" is the baseline size divided by the variant size. Pruned rows without "+ INT8" are dynamic-range INT8 models.

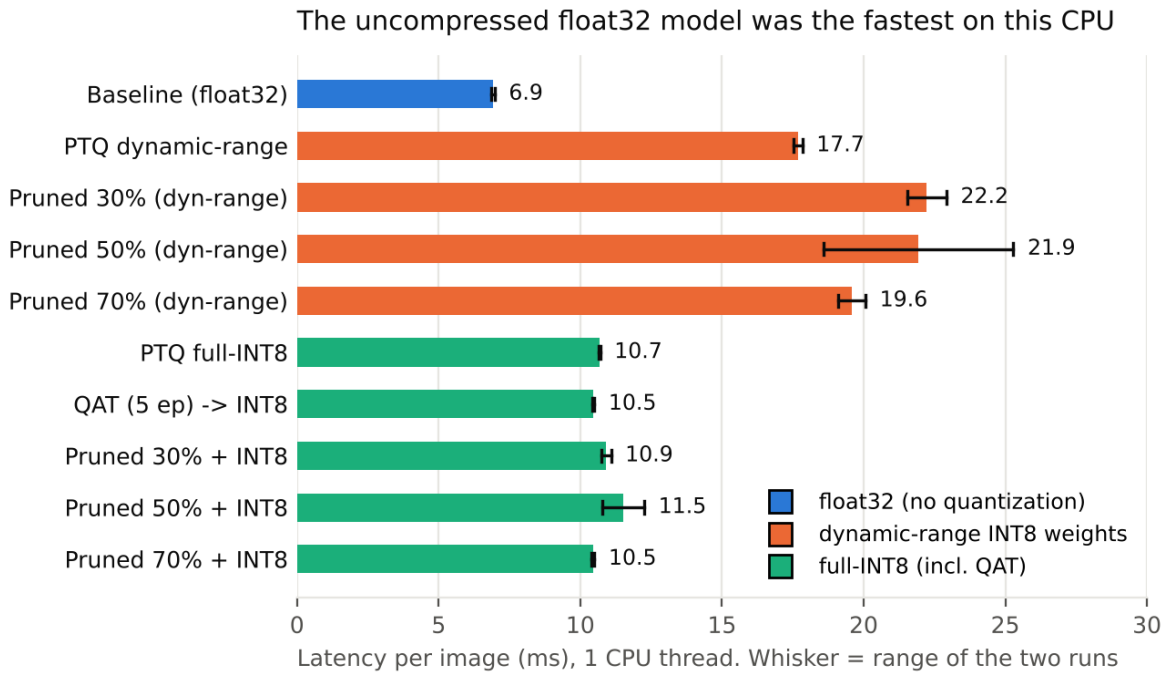


Figure 1. Latency. Colour marks the kind of model, not the pruning level.

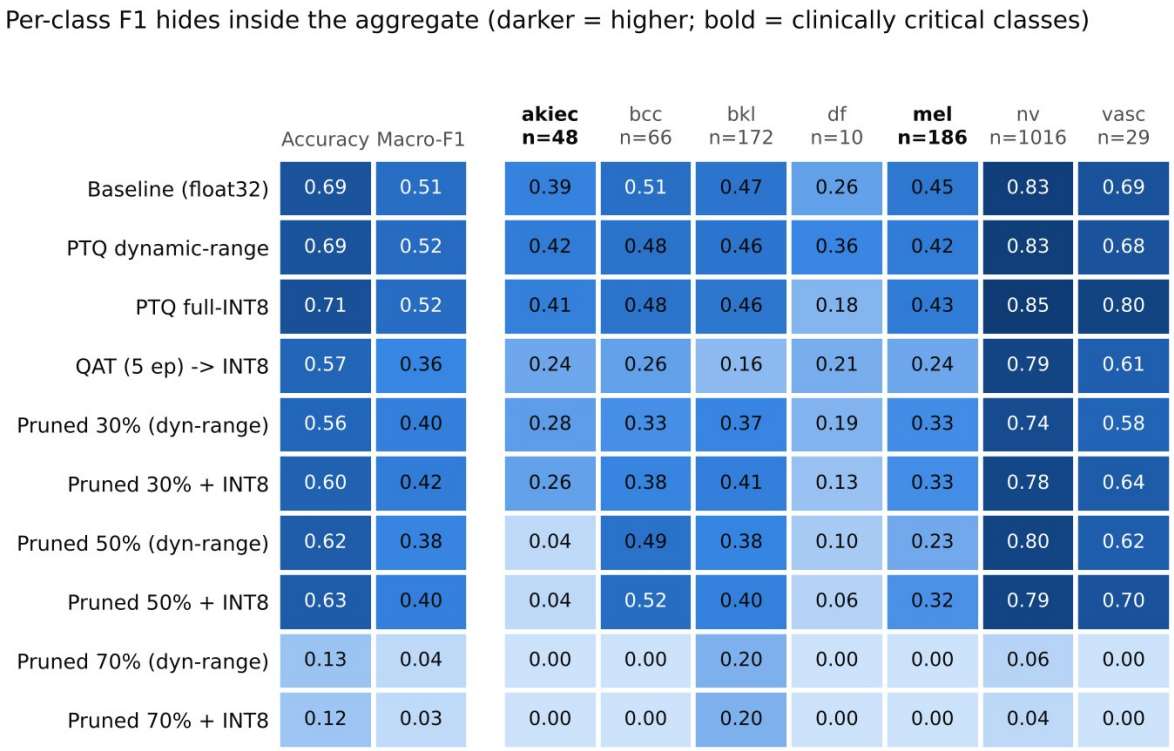


Figure 2. Accuracy and F1 per class. In the 50% rows, akiec F1 is near zero while the aggregate columns still look usable.

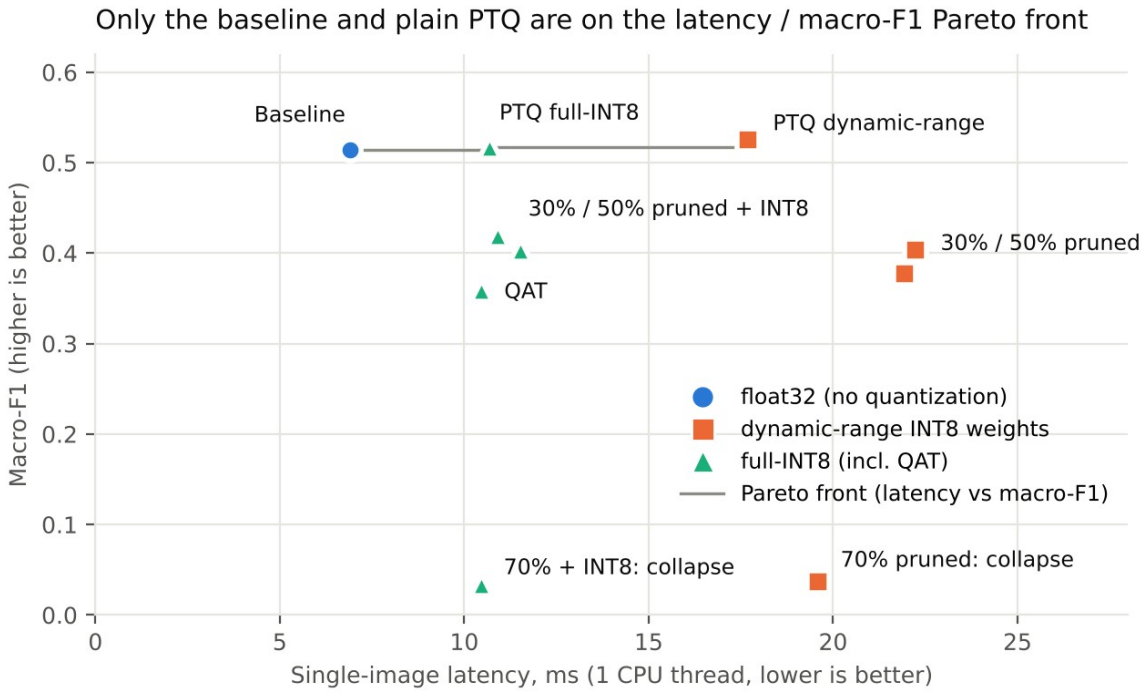


Figure 3. Latency against macro-F1. The grey line joins the non-dominated variants.

4.3 What the numbers say

Finding 1. INT8 PTQ lost nothing measurable

Full-INT8 is 3.3× smaller with accuracy 0.713 and macro-F1 0.517; dynamic-range is 3.5× smaller with 0.687 and 0.525. mel F1 moved from 0.446 to 0.431 and 0.421, akiec F1 from 0.389 to 0.408 and 0.421. With 1,527 test images the 95% interval on accuracy is about ±2.3 points, and on recall for mel (n = 186) about ±7 points, so none of these differences is separable from noise. The +0.028 accuracy of full-INT8 mostly comes from the

majority class (nv F1 0.830 to 0.855), so it should not be read as an improvement. **Conclusion:** at this scale and on this test set, INT8 PTQ keeps quality at about a third of the size.

Finding 2. Aggregate accuracy hid a class-level collapse under pruning

Change vs baseline	Accuracy	Macro-F1	mel F1	akiec F1
Pruned 30% (dyn-range)	−0.125	−0.111	−0.112	−0.113
Pruned 30% + INT8	−0.088	−0.095	−0.113	−0.126
Pruned 50% (dyn-range)	−0.062	−0.137	−0.217	−0.352
Pruned 50% + INT8	−0.050	−0.111	−0.125	−0.352
PTQ dynamic-range	+0.002	+0.010	−0.025	+0.032
PTQ full-INT8	+0.028	+0.002	−0.015	+0.019
QAT (5 ep) -> INT8	−0.118	−0.156	−0.207	−0.153

Table 6. Change relative to the baseline (negative = worse).

Going from 30% to 50% sparsity, accuracy went up (0.560 to 0.623) while akiec F1 went from 0.276 to 0.036; the model drifted toward predicting nv, which raises accuracy and kills the rare class. This is the failure per-class tracking was meant to catch: a pruning level chosen from an accuracy target would pass 50% and silently stop detecting akiec. akiec has 48 test images, so a drop from 0.389 to 0.036 is far outside noise.

Why this may not generalise. Even 30% unstructured pruning cost 12.5 accuracy points, which is more than typically expected for MobileNetV2 and points at the recovery recipe rather than the sparsity: 8 epochs at lr 1e-5, class weights without the mel/akiec boost used for the baseline, uniform sparsity in every layer including the sensitive early and depthwise ones, and final-epoch weights with no checkpoint selection. These causes were not isolated.

Finding 3. 70% sparsity produced a failed model, not a degraded one

Accuracy 0.126 and macro-F1 0.037. Five classes have F1 = 0 (akiec, bcc, df, mel, vasc), bkl is 0.198 and nv 0.058. That is worse than always predicting nv (accuracy 0.665), so the model is broken. The cause is unknown; the same recipe caveats apply.

Finding 4. Smaller was not faster on this CPU, and pruning bought neither size nor speed

Float32 baseline: 6.9 ms. Full-INT8 variants: 10.5–11.5 ms (1.5–1.7× slower). Dynamic-range variants: 17.7–22.2 ms (2.6–3.2× slower). The ordering was identical in both benchmark runs. Run-to-run spread was under 2% for the baseline and PTQ models but 30% for pruned 50%, so single timings on a shared Colab VM are noisy and the ordering is more reliable than the exact values. File size was identical across the three pruning levels (2,447.5 KB) because the stored arrays are dense: zeros are written out like any other value. Dense kernels also ignore zeros, which fits the flat latency.

Hypothesis, not measured: float32 kernels on this CPU are heavily optimised (TFLite's XNNPACK path), while INT8 execution on x86 either lacks equivalent kernels or adds quantize/dequantize overhead. If true, the ranking could reverse on ARM CPUs with integer instructions, which is where such models are usually deployed. Two cheap tests are listed in Section 7.

Finding 5. QAT underperformed, probably because it was under-trained

Macro-F1 0.359 against 0.517 for plain PTQ. QAT ran for only 5 epochs at lr 1e-5 with the unboosted class weights. I read this as an unfinished experiment, not as evidence that QAT is worse than PTQ.

Pareto summary

On latency vs macro-F1, the non-dominated variants are the baseline, PTQ full-INT8 and PTQ dynamic-range. On latency vs akiec F1 it is the same three. On latency vs mel F1 only the baseline is non-dominated. Their macro-F1 values (0.515, 0.517, 0.525) are indistinguishable, so in practice nothing compressed beat the baseline on speed and quality together, and the reason to quantize here is memory and storage.

5. How this fits with prior work

The baseline accuracy (68.5%) is well below paper 1 (83.15%) and the other published figures. I would not read this as a clean gap. I do not know whether those papers split by lesion, and an image-level split alone can add many points on HAM10000. My training budget was also short (early stopping, no tuning beyond three versions). What is solid is the within-study comparison: every variant here shares the same data, split and baseline.

On the three things this report adds: per-class tracking produced the clearest result (Finding 2); the latency result stands as a measurement on one machine; the full-sweep framing is real but its novelty depends on paper 5's full text.

6. Limitations

1. **One seed, one split, one run per model, no confidence intervals.** Differences below roughly 2–3 accuracy points or 7 points of mel recall are within sampling noise. I did not measure training-to-training variance, so the small gaps between baseline versions v1, v2 and v3 may be noise too.
2. **Baseline v3 was selected using test-set results.** The effect is probably small, but it is not zero.
3. **Compression stages are confounded with the baseline's training recipe.** They used unboosted class weights, a low learning rate and final-epoch weights. Pruning and QAT results show what this recipe does, not what the techniques can do.
4. **Pruning is unstructured and no sparse runtime was used.** Channel-level (structured) pruning, which shrinks dense models, was not tested.
5. **Latency is from one shared Colab CPU, not recorded by model or architecture, and not from ARM.** One fixed input image, 100 timed calls, and a 30% spread for one model. It says nothing about phones, Raspberry Pi or microcontrollers.
6. **Small classes are noisy:** df has 10 test images, vasc 29, akiec 48.
7. **The literature scan is abstract-level**, so novelty statements are provisional.
8. **Not a clinical study.** One dataset from two sites, no external validation. The models must not be used for diagnosis.

7. What to do next, cheapest first

Under an hour each

- Record the Colab CPU (lscpu) and TensorFlow version, and repeat the latency benchmark five times; report median and range.
- Run the float32 baseline with XNNPACK disabled. If it slows to the INT8 range, the Finding 4 explanation holds.
- Evaluate pruned_30.keras directly (before TFLite conversion) to separate the effect of pruning from the effect of conversion.
- Save per-image predictions in evaluate.py so bootstrap confidence intervals can be computed for every variant.

A few hours on Colab GPU

- Re-run pruning with the boosted class weights, a higher recovery learning rate or more epochs, and finer sparsity steps (10 to 50%) to find where the cliff actually is.
- Re-run QAT for 15–20 epochs.
- Train the baseline with three seeds to measure training variance.

Larger

- Measure latency on an ARM device (Raspberry Pi or an Android phone). This matters most, since ARM is the realistic deployment target.
- Try structured (channel) pruning, which reduces dense compute.
- Read paper 5 in full and update Section 2.

8. Project log and reproducibility

- **Planning:** chose skin-lesion compression with a latency and per-class focus after the literature scan; wrote the pipeline and tested it end to end on synthetic data.
- **Training:** on a Colab GPU. Three baseline versions (Table 2). Colab runtime resets wiped saved models, so models were re-uploaded or regenerated.
- **Compatibility fix:** tfmot needs legacy Keras 2, while the baseline had been saved in Keras 3 format. The weights were extracted, the model rebuilt under legacy Keras, and re-saved. The float32 TFLite built from it matched the original evaluation exactly.
- **Compression:** PTQ, pruning at 30/50/70% (each with INT8), QAT. PTQ files were regenerated after the session loss, and a float32 TFLite baseline was added so the benchmark has an uncompressed reference.
- **Benchmark:** run twice; results in Appendix A.
- **Code:** github.com/vijay1011-stack11/edge-skin-lesion-compression (pipeline, notebooks, literature scan and results).

References

[1] Chaturvedi, Gupta, Prasad. Skin Lesion Analyser: An Efficient Seven-Way Multi-Class Skin Cancer Classification Using MobileNet. 2019. arXiv:1907.03220.

[2] Baig et al. Light-Dermo: A Lightweight Pretrained CNN for the Diagnosis of Multiclass Skin Lesions. Diagnostics 13(3):385, 2023.

[3] Al Mamun et al. Optimizing Deep Learning for Skin Cancer Classification: A Computationally Efficient CNN with Minimal Accuracy Trade-Off. 2025. arXiv:2505.21597.

[4] Wang et al. Quantization-Aware Neuromorphic Architecture for Efficient Skin Disease Classification on Resource-Constrained Devices. 2025. arXiv:2507.15958.

[5] Winata, Andryani, Gunawan, Lumban Gaol. Efficiency Analysis of AI Model Compression for Edge Teledermatology. Studies in Computational Intelligence, vol. 1227, Springer, 2026. (abstract only)

[6] Paxton et al. Enhancing Fairness in Skin Lesion Classification for Medical Diagnosis Using Prune Learning. 2025. arXiv:2509.00745.

[7] Tschandl, Rosendahl, Kittler. The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions. Scientific Data 5:180161, 2018.

[8] Sandler et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR 2018.

Appendix A. Full tables

A1. F1 per class

Model	akiec (n=48)	bcc (n=66)	bkl (n=172)	df (n=10)	mel (n=186)	nv (n=1016)	vasc (n=29)
Baseline (float32)	0.389	0.513	0.468	0.261	0.446	0.830	0.694
PTQ dynamic-range	0.421	0.484	0.465	0.364	0.421	0.835	0.685
PTQ full-INT8	0.408	0.477	0.464	0.185	0.431	0.855	0.800
QAT (5 ep) -> INT8	0.236	0.263	0.164	0.214	0.239	0.786	0.609
Pruned 30% (dyn-range)	0.276	0.330	0.371	0.189	0.334	0.741	0.583
Pruned 30% + INT8	0.263	0.379	0.411	0.131	0.333	0.778	0.638
Pruned 50% (dyn-range)	0.036	0.488	0.375	0.095	0.229	0.795	0.622
Pruned 50% + INT8	0.036	0.520	0.396	0.063	0.322	0.794	0.696
Pruned 70% (dyn-range)	0.000	0.000	0.198	0.000	0.000	0.058	0.000
Pruned 70% + INT8	0.000	0.000	0.198	0.000	0.000	0.040	0.000

Table A1. F1 for every class and variant. Support (n) is the number of test images in the class.

A2. Latency, both benchmark runs

Model	Run 1 (ms)	Run 2 (ms)	Spread between runs
Baseline (float32)	6.977	6.854	1.8%
PTQ dynamic-range	17.529	17.837	1.7%
PTQ full-INT8	10.702	10.655	0.4%
QAT (5 ep) -> INT8	10.498	10.437	0.6%
Pruned 30% (dyn-range)	21.529	22.924	6.3%
Pruned 30% + INT8	10.737	11.086	3.2%
Pruned 50% (dyn-range)	25.288	18.593	30.5%
Pruned 50% + INT8	12.260	10.776	12.9%
Pruned 70% (dyn-range)	20.069	19.102	4.9%
Pruned 70% + INT8	10.402	10.500	0.9%

Table A2. Spread = difference between the runs as a share of their mean.

Appendix B. How the noise figures were estimated

Intervals are 95% normal approximations to a binomial proportion, $1.96 \cdot \sqrt{p(1-p)/n}$: accuracy with $p \approx 0.69$ and $n = 1{,}527$ gives ±2.3 points; mel recall with $p \approx 0.45$ and $n = 186$ gives ±7 points; akiec recall with $p \approx 0.40$ and $n = 48$ gives ±14 points. They are rough guides, not bootstrap intervals, and they ignore that all models are evaluated on the same images.